

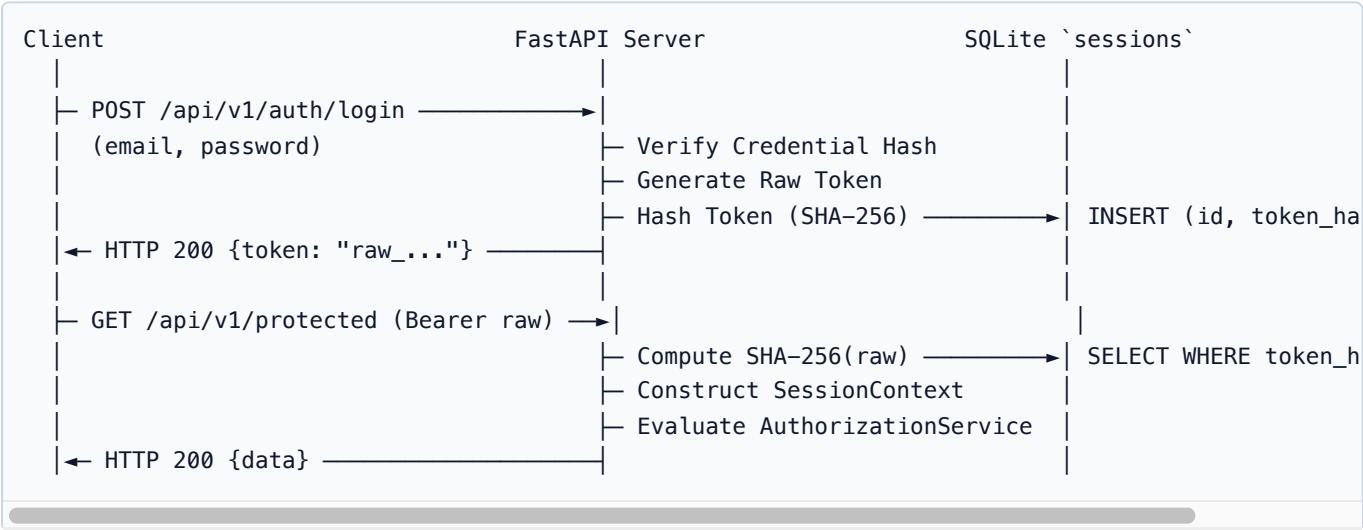
Navigators Technical Documentation

Navigators — Authentication & Role-Based Access Control (RBAC)

Document Identifier: RBAC-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

1. Authentication Architecture

Navigators implements a cryptographically secure, stateless bearer-token session model: - **Token Generation:** On registration or login, the server generates a 32-byte cryptographically secure random token (`secrets.token_urlsafe(32)`). - **One-Way Token Storage:** The raw token is delivered strictly once to the client in the HTTP response body. The backend computes the SHA-256 digest (`hashlib.sha256(token.encode()).hexdigest()`) and records only the hash in the `sessions` table. - **Session Lifecycle:** Sessions expire automatically after 30 days (`expires_at`), or immediately upon explicit logout (`revoked_at IS NOT NULL`).



2. The Seven-Tier Role Hierarchy

The platform defines seven discrete roles with strictly partitioned operational capabilities:



3. RBAC Permission Matrix

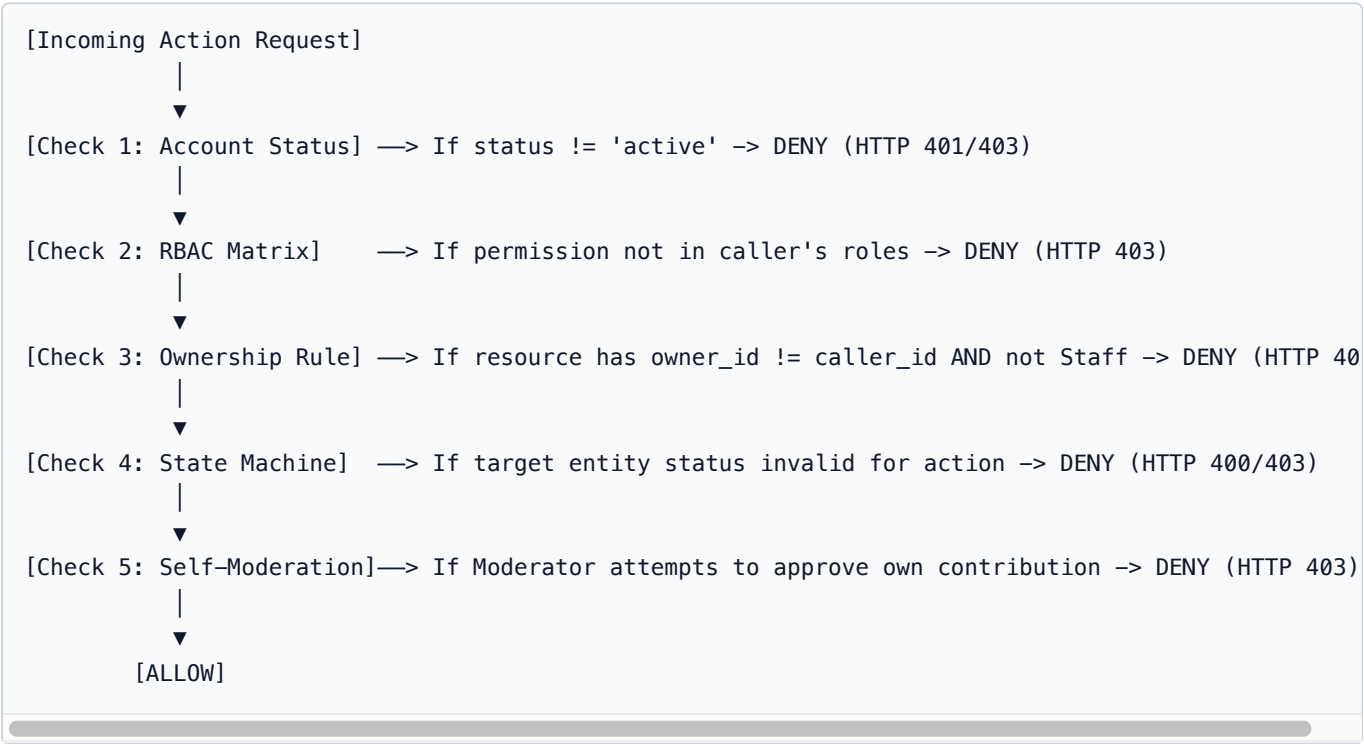
The database defines 40+ atomic permissions mapped through `role_permissions` :

Resource	Action	Permission Identifier	Min Role Required	Description
place	read	place:read	guest	Read verified places and canonical road network.
place	create	place:create	moderator	Publish verified place directly into canonical map.
place	update	place:update	moderator	Modify existing canonical place attributes.
place	delete	place:delete	team_admin	Soft-delete canonical place.
contribution	create	contribution:create	user	Author new POI or map correction draft.
contribution	read	contribution:read	user	View public/own contributions.
contribution	update	contribution:update	user	Edit owned draft contributions.
contribution	withdraw	contribution:withdraw	user	Cancel owned pending submission.
contribution	approve	contribution:approve	moderator	Approve pending submission into map.
contribution	reject	contribution:reject	moderator	Reject pending submission with feedback notes.
dataset	upload	dataset:upload	internal_contributor	Ingest new vehicle sensor recording.
dataset	validate	dataset:validate	team_admin	Certify recorded session for model training.
model	read	model:read	user	Inspect deployed active model metadata.
model	train	model:train	team_admin	Launch PyTorch training job.

Resource	Action	Permission Identifier	Min Role Required	Description
model	deploy	model:deploy	team_admin	Promote candidate model to production.
model	rollback	model:rollback	team_admin	Revert production model to previous version.
audit	read	audit:read	team_admin	Inspect system audit trails and compliance logs.
user	manage_roles	user:manage_roles	super_admin	Assign or revoke roles for any account.

4. Multi-Layered Authorization Engine (`AuthorizationService`)

Permissions alone do not guarantee access. The central `AuthorizationService` (`src/db/authorization.py`) enforces a multi-tiered validation pipeline:



Self-Moderation Prevention:

A moderator cannot review their own submissions. If `resource.owner_id == caller.user.id`, approval and rejection actions are blocked with `PERMISSION_DENIED` to prevent conflict of interest.